Reinforcement Learning for AIE (708.006), SS26
Assignment 1
Monte Carlo Simulation, Markov Decision Process, Iterative Policy Evaluation

Alexander Mayr
alexander.mayr@tugraz.at

Teaching Assistant:	Leon Tiefenböck
Points to achieve:	20 pts
Deadline:	~~17.04.2026 23:59~~ 01.05.2026 23:59
Hand-in procedure:	This is a **team assignment** recommended for teams of 2 students. Submit **your report (PDF)** and your Python files (`MonteCarloEstimation.py/ipynb`, `twostatemachine.py` and `policy_iteration.py`) to the TeachCenter. Upload just these files. Do not upload a folder. Do not zip them. Only one submission per group.
Plagiarism:	If detected, 0 points for all parties involved. If this happens twice, we will grade the group with "Ungültig aufgrund von Täuschung"
AI usage:	For this Assignment use of AI tools like Co-Pilot and ChatGPT is not allowed. If proven the group will receive 0 pts.

General Remarks

- All results (i.e., plots, results of computations) should be included in your PDF report – the Python / Jupyter Notebook files should only serve as a way to reproduce your results.
- For all pen-and-paper exercises, report all intermediate steps – only presenting the final solution is insufficient.

Task 1 – Monte Carlo Simulation [4 Points]

A random variable X with state space $\mathcal{X} = \{0, 1, 2, 3, 4\}$ has the following probability mass function[1]

$$p(X = k) = \begin{cases} M \cdot \frac{\lambda^k e^{-\lambda}}{k!} & \text{for } k = 0, 1, 2, 3, 4 \\ 0 & \text{otherwise} \end{cases}$$

where $k!$ is the *factorial* of k and M is a normalizing constant. Let $\lambda = 4$.

a) Analytically derive the normalizing constant M such that $\sum_{k \in \mathcal{X}} p(X = k) = 1$.

b) Analytically compute $\mathbb{E}_X[X]$.

[1] Note that this PMF is essentially a *truncated Poisson* distribution.

c) Use Monte Carlo simulation to approximate the expected value of X and show a convergence plot with the y-axis representing the estimated expected value and the x-axis the number of simulations utilized to calculate the estimated expected value. Use $100, 200, 300, \ldots, 10000$ simulations and obtain an estimate for each value. Draw a horizontal line representing the exact expectation. Include this plot in your report.

Task 2 – Markov decision process [6 Points]

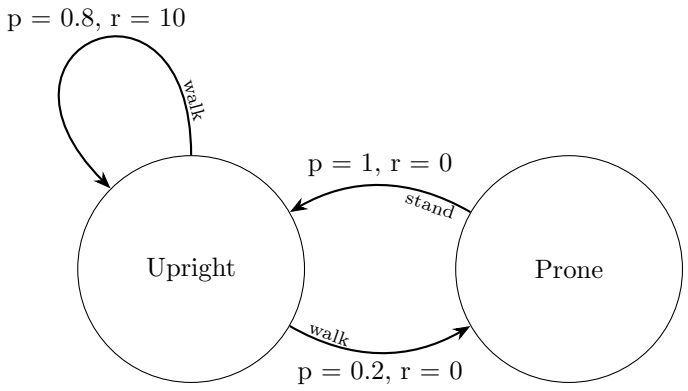


Figure 1: A simple two-state robot: upright (s_1) and prone (s_2). When upright, it walks - staying upright with 0.8 probability or falling prone with 0.2. When prone, it must stand to return upright (probability 1.0). Only staying upright ($(s_1) \to (s_1)$) yields a reward of 10; all other transitions give 0.

Imagine a simple toy robot with two states: upright (s_1) and prone (s_2), and two actions: walk and stand.

When upright, it can use the walk action: it stays upright with probability 0.8 or falls prone with probability 0.2. When prone, it cannot use walk and must use the action stand, which always returns it to upright (probability 1.0).

Each state allows only one action, so the system can be modelled as a Markov chain. The only rewarded transition is staying upright ($s_1 \to s_1$), which yields a reward of 10; all other transitions have reward 0.

Our goal is to determine the long-term expected reward (value) of each state in the Markov chain—specifically, the rewards accumulated if the process runs indefinitely, alternating between the *Upright* and *Prone* states as represented in Figure 1.

This is typically approached using the **Bellman equation**, which in its general form is:

$$v(s) = \max_{a \in A} \sum_{s' \in S} P(s, a, s') \left[R(s, a, s') + \gamma v(s') \right]$$

However, for this simple example, we do not need value iteration. Since there is only one possible action per state, the Bellman equation simplifies to:

$$v(s) = \sum_{s' \in S} P(s, s') \left[R(s, s') + \gamma v(s') \right]$$

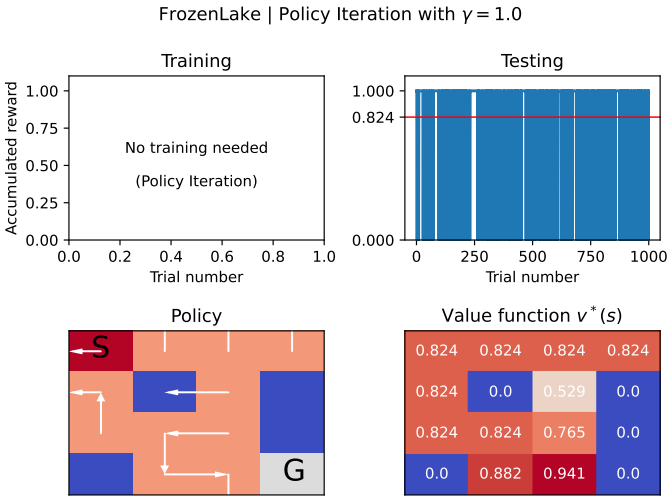
a) Analytically derive the values $v(s_1)$ and $v(s_2)$ from the simplified Bellman equation using $\gamma = 0.5$.

b) Implement the TODOs in `template_twostatemachine.py`. Try to understand the code fully. For testing purposes `test_twostatemachine.py` is provided, add your analytic solution to test 6 for comparison. Do not forget to rename your own files accordingly.

Task 3 – Planning in a Grid World [10 Points]

A grid world is a typical environment with finite action and state spaces. We will solve the FrozenLake[2] environment from the `gymnasium` package (a fork of OpenAI's Gym package). The agent controls the movement of a character in a grid world. Some tiles of the grid are walkable, and others lead to the agent falling into the water. Additionally, the movement direction of the agent is uncertain and only partially depends on the chosen direction (the lake is slippery). The agent receives a reward of 1 if it moves to the goal state and 0 else. The episode ends as soon as the agent falls into the water or reaches the goal state. Further information can be found in the Gymnasium Documentation.

(a) FrozenLake environment. The starting state is on the top left while the goal state is on the bottom right.



(b) Value function and policy that were acquired using *Policy Iteration* ($\gamma = 1$). Testing the policy using 1000 episodes yields an average sum of rewards of 0.824.

a) Assume we know the dynamics of the underlying MDP (i.e., the state transition probabilities and the expected rewards). Fill in the TODOs in `policy_iteration.py`: You are supposed to implement the *Policy Iteration* algorithm, which consists of repeatedly *evaluating a policy* (using *Iterative Policy Evaluation*) and improving this policy by acting greedily w.r.t. the Q function of the old policy. When finished with the TODOs, execute the file: This will run policy iteration until convergence for both the undiscounted case ($\gamma = 1$) and a discounted case ($\gamma = 0.95$). In both scenarios, the code framework will produce a plot which includes (1) the policy you have found, (2) the corresponding value function, and (3) the reward that was accumulated in 1000 simulated episodes (and its average). Include both plots ($\gamma = 1$ and $\gamma = 0.95$) in your report. Explain any differences between the two policies, the two value functions, and between the average test success rate (i.e., the fraction of simulated episodes in which the agent reached the goal state). Briefly discuss why these differences appear.

b) Assume that s_{start} is the starting state. In the undiscounted case ($\gamma = 1$), what is the relationship between $v^*(s_{\text{start}})$ and the average reward you encounter when acting according to π^* for N episodes (i.e., `np.mean(policy_iteration.test_policy(num_episodes=`N`))`)? What happens as $N \to \infty$? Write down the definition of the v function in this particular case ($\gamma = 1$, binary reward per episode).

c) In general: Is the optimal policy π^* unique? Is the optimal value function v^* unique? Explain your reasoning.

[2] `https://gymnasium.farama.org/environments/toy_text/frozen_lake/`